From Tracking to Deployment: Managing ML Experiments with MLflow

Open source tools like MLflow help teams maintain a disciplined ML lifecycle without relying on fully managed platforms.

In every real-world machine learning project, the experiments multiply before you realise it. You tweak hyperparameters, change datasets, try different models -- and suddenly you're juggling dozens of runs.

Soon the questions start creeping in:

- Which model did we ship last month?
- What parameters gave the best accuracy?
- Why is the production model behaving differently from our training version?

This chaos isn't a 'data science problem'. It's an 'experiment management problem', and MLflow solves exactly that.

MLflow provides a simple, open source way to track experiments, compare results, version models, and automate the path from training to deployment. With one tool, teams get transparency, reproducibility, and governance across the entire machine learning (ML) lifecycle.

What MLflow is — and the problem it solves

MLflow is not a model training framework. It is the glue around your ML workflow.

It provides four core components:

- ***Tracking***: Logs parameters, metrics, artifacts
- ***Projects:*** Package code in a reproducible format
- ***Models:*** Standardise the way you store and serve models
- ***Model Registry***: Central hub for versioning and promoting models

MLflow's architecture is shown in Figure 1.

Setting up MLflow

MLflow can be used:

- Locally (simple experiment tracking)
- On-prem (custom ML platforms)
- On cloud environments (Databricks,

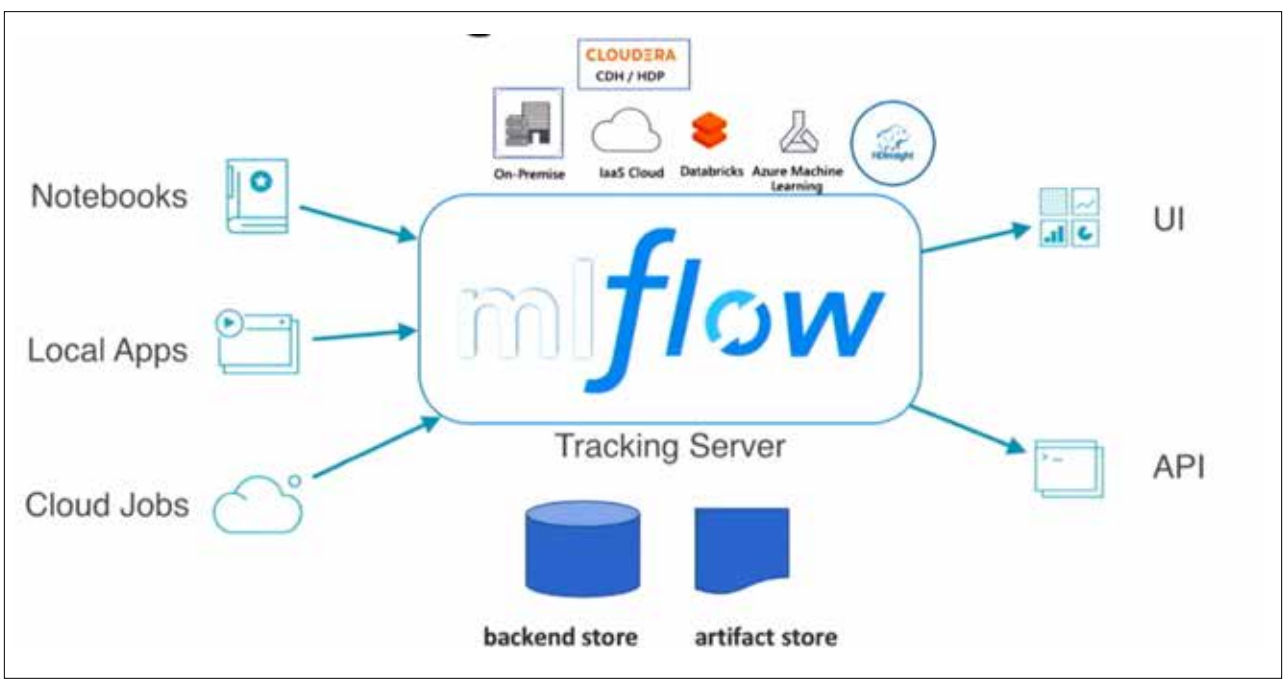

Figure 1: MLflow architecture – tracking server, backend store, and artifact store
(Source: *https://github.com/amitgoswami1027/ProductionalizeMachineLearningModels-Master*)